

実験ログ: 剛体と破壊 (RBD)

2026-09-24 | 収録 7 件

Houdini 研究部で、hython で実際に測った実験の記録。数字はどれも測った値。

収録している実験

- ・ RBD 破壊 — 最初の本物のシミュレーション
- ・ ドミノを倒す — 間隔は高さの 0.25~1.03倍。初速は点に付けないと効かない
- ・ 壁を崩す — 拘束の強さの窓は狭い。弱いと自重で崩れ、強いとぶつけても壊れない
- ・ voronoifracture は種 N 個で N 個のかげら、体積の合計は元のまま — 切り口を作らないと、中のかげらは消える
- ・ 種を relax すると、voronoifracture のかけらはそろうどころかばらつく — 種が箱の壁へ押し出される
- ・ rdbbulletsolver の箱は地面ぴったりで止まる — Collision Padding を変えても止まる高さは同じ、当たった瞬間だけ少し沈む
- ・ rdbbulletsolver の沈み込みは Bullet Substeps で決まる — 1 だと 0.38 めり込んで沈んだまま、50 でほぼ 0

RBD 破壊 — 最初の本物のシミュレーション

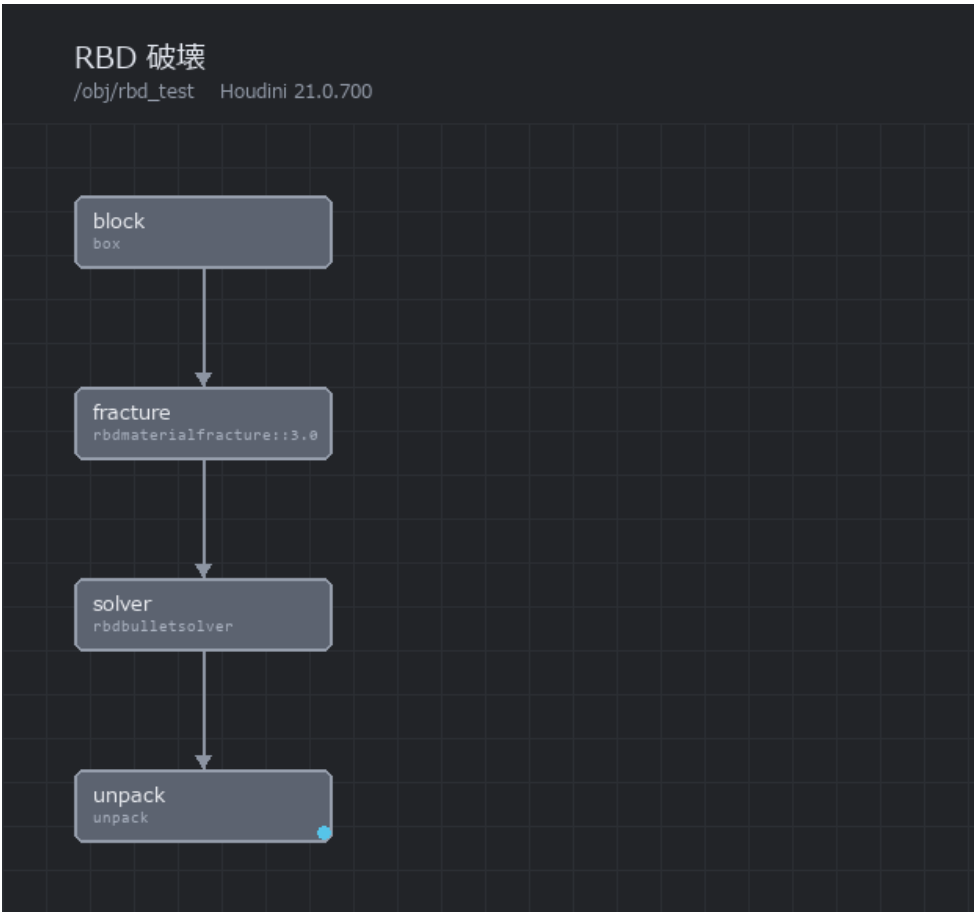
2026-09-24 | Houdini 21.0.700 | /obj/rbd_test | ノード 4 / 接続 3

何を作ったか

実験013で、剛体（RBD）は SOP レベルのノードが47種あり、パーティクルより入口が易しいと分かった。ここまでの実験はすべて「1フレームの計算」だったが、今回は前のフレームの結果を次に渡していく本物のシミュレーション。

シミュレーションは結果が複雑で、目で見ても正しいかどうか判断しづらい。そこで検証の軸を1つ決めた。剛体は変形しないので、砕けても落ちてもぶつかっても、全部の破片の体積の合計は変わらないはず。これが崩れたら計算のどこかがおかしい。

ノードの接続



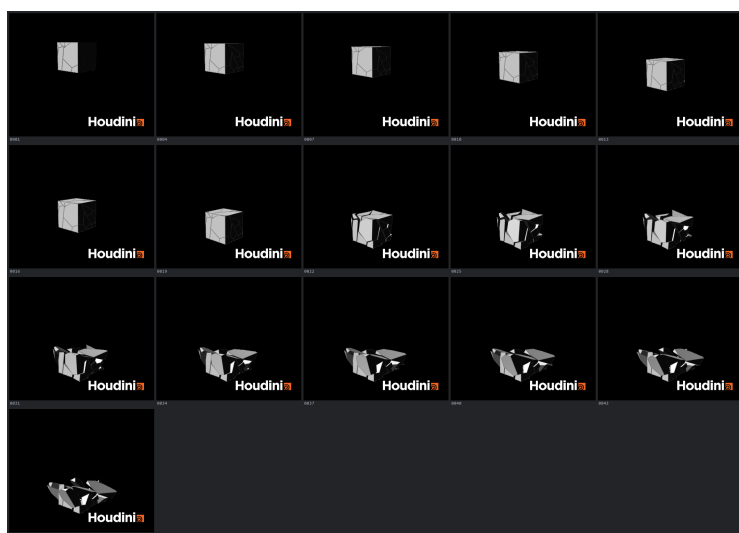
スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
block	box	scale = 2.0 ty = 4.0
fracture	rbdmaterialfracture::3.0	(変更なし)

ノード	タイプ	デフォルトから変更したパラメータ
solver	rbdbulletsolver	folder10 = 1 folder13 = 1 folder21 = 1 folder24 = 1 folder26 = 1 folder26_1 = 1 folder26_2 = 1 folder27 = 1 folder3 = 1 folder34 = 1 folder35 = 1 folder38 = 1 …ほか 11 件
unpack	unpack	(変更なし)

高さ4から落として砕く（16コマ）

2×2×2 の箱を rbdmaterialfracture（コンクリート）で25個の破片に砕き、rbdbulletsolverで落とした。地面はソルバの機能で用意している。



16コマ。落下中は塊のまま、着地して初めて砕ける

フレームごとの状態

「横の広がり」はバウンディングボックスの水平方向の大きさ。元の箱は 2.0 なので、これが増えていけば破片が散らばったということ。

フレーム	体積の合計	最下点	横の広がり	高さ	計算にかかった秒
1	8.00000	3.0000	2.0000	2.0000	0.38
4	8.00000	2.9208	2.0000	2.0000	0.02
10	8.00000	2.3028	2.0000	2.0000	0.03
20	8.00000	-0.0492	2.0589	2.0331	0.05
30	8.00000	-0.0225	3.7483	2.0181	0.03
46	8.00000	-0.0219	5.6408	1.9498	0.05

検証: 体積は保たれているか

調べたこと	結果
元の箱 (2×2×2) の体積	8.00000
砕いた直後の合計	8.00000 (差 0.00000)
全フレームでの振れ幅	0.0000000 (0.0000%)

わかったこと

- ・体積は完全に保たれた。全6フレームで 8.00000 ちょうど、振れ幅は 0.0000000。元の箱の体積 8.0 と完全に一致する。砕いても落としてもぶつかっても体積が変わらないので、剛体シミュレーションとして正しく動いている。
- ・これは目で見ても分からない種類の検証。破片が散らばった画を見ても「正しく計算できているか」は判断できないが、体積が保たれていれば少なくとも物質は増減していないと言える。実験012で boolean に使った検算と同じ考え方。
- ・落下中は砕けない。フレーム1から10まで、横の広がり 2.0000 のまま完全に変化しない。砕いてはあるが、破片はくっついたまま1つの塊として落ちる。
- ・着地の瞬間に広がり始める。フレーム20で最下点が -0.0492 になり（地面を少し突き抜ける）、同時に広がりが 2.0589 に増える。その後 30で 3.7483、46で 5.6408。元の 2.8倍まで散らばった。
- ・同時に全体の高さは 2.0000 から 1.9498 に下がる。積み上がっていたものが平たく崩れたということ。
- ・最下点は着地時に -0.0492 まで沈み、その後 -0.0219 で落ち着く。破片は地面を少し突き抜けたまま止まる。ソルバが衝突判定に持たせている余裕（マージン）によるもので、厳密に地面の上に乗るわけではない。
- ・rbdmaterialfracture のコンクリートモードで、2×2×2 の箱が 25個の破片になった（282点 / 191面）。破片の区別は name というプリミティブ属性で管理されている。
- ・計算は速かった。最初のフレームが 0.38秒、以降は 0.02～0.05秒。全部で1秒程度。シミュレーションといっても、この規模なら実験を回すのに支障はない。
- ・rdbbulletsolver の既定は substeps = 10、cacheenabled = 1。1フレームを10回に分けて計算しており、結果は自動でキャッシュされる。2回目以降のフレーム送りが速いのはこのため。
- ・地面はソルバの useground で用意できる。別途ジオメトリを作る必要はない。

次にやること

- ・substeps を減らすと結果がどう変わるか。突き抜けや不安定さが出るはず
- ・破片の数を増やしたときの計算時間の伸び方を測る
- ・Vellum（布）でも同じように、保存されるべき量を見つけて検証する

ドミノを倒す — 間隔は高さの 0.25～1.03倍。初速は点に付けないと効かない

2026-09-24

何を作ったか

高さ 0.6・厚み 0.08・幅 0.3 の板を等間隔に並べ、最初の1枚に横向きの初速 1.6 を与えて倒した。板は copytopoints で配って Pack and Instance にし、rdbbulletsolver で解く。

最初はまったく動かなかった。初速 v をプリミティブ（1枚=1つの塊）に付けていたのが原因で、点に付け直すと動いた。名前（name）の有無は関係なく、name を付けても付けなくても40フレーム目の位置は3桁まで同じだった。

間隔には上下の境目があった。狭すぎても倒れない。間隔 0.15（高さの 0.25倍）では12枚中3枚で止まる。狭み撃ちで詰めると、境目は 0.15～0.1531（高さの 0.25～0.26倍）。広いほうは、0.62（高さの 1.03倍）までは最後まで倒れ、0.66（1.10倍）で2枚で止まった。

倒れていく速さは、間隔が広いほど遅い。12枚が倒れきるまでの時間は、間隔 0.25 で 1.042秒（1秒に10.56枚）、0.35 で 1.875秒（5.87枚）、0.45 で 2.833秒（3.88枚）、0.55 で 3.875秒（2.84枚）、0.62 で 5.375秒（2.05枚）。

枚数を変えても、1秒あたりの枚数はだいたい同じだった。間隔 0.3 で 6枚 10.00枚/秒、12枚 7.33枚/秒、24枚 6.57枚/秒。枚数が多いほど少し遅くなる。

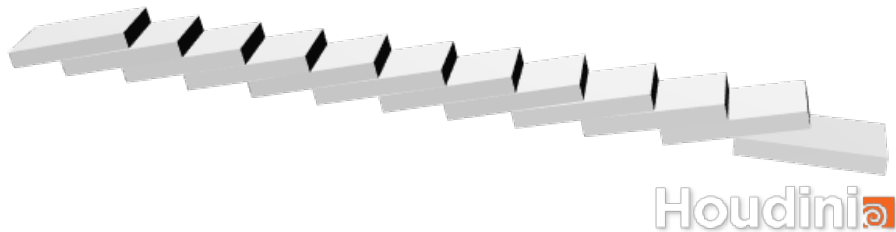
最初は96フレームで打ち切っていたので、24枚と広い間隔が「途中で止まった」ように見えた。160フレームまで伸ばすと最後まで倒れた。

倒れていくところ（12枚・間隔 0.3）

2枚とも同じカメラ。左端の1枚に初速 1.6 を与えただけで、あとは当たって倒れていく。



30フレーム目。倒れた列と立っている列の境目が進んでいく。



60フレーム目。12枚すべてが倒れている（最後の1枚は53フレーム目）。

A. 間隔を変える（12枚・160フレームまで）

「倒れた」は、板の上向きがまっすぐ上から60度以上傾いたとき。時間は最初の1枚が倒れてから最後の1枚が倒れるまで。

間隔	高さとの比	倒れた枚数	かかった時間	1秒あたり	計算の秒
0.15	0.25倍	3 / 12	0.125秒	—	0.81
0.25	0.42倍	12 / 12	1.042秒	10.56枚	0.53
0.35	0.58倍	12 / 12	1.875秒	5.87枚	0.53
0.45	0.75倍	12 / 12	2.833秒	3.88枚	0.55
0.55	0.92倍	12 / 12	3.875秒	2.84枚	0.55
0.58	0.97倍	12 / 12	4.333秒	2.54枚	0.78
0.62	1.03倍	12 / 12	5.375秒	2.05枚	0.50
0.66	1.10倍	2 / 12	0.458秒	—	0.47
0.70	1.17倍	2 / 12	0.750秒	—	0.47

B・C. 境目と枚数

境目は挟み撃ちで5回詰めた。枚数を変えた行は間隔 0.3。

調べたこと	結果
狭いほうの境目	間隔 0.15～0.1531（高さの 0.25～0.26倍）
広いほうの境目	間隔 0.62～0.66（高さの 1.03～1.10倍）
6枚	0.500秒で全部・1秒に 10.00枚
12枚	1.500秒で全部・1秒に 7.33枚
24枚	3.500秒で全部・1秒に 6.57枚

わかったこと

- ・初速は点に付ける。プリミティブに付けても効かない。packed の塊に v@v を書いても, rdbbullet solver は動かさなかった。setpointattrib(0, "v", pts[0], ...) と点に書くと動く。エラーも警告も出ない。「黙って無視されるパラメータ」の仲間。
- ・ name の有無は関係なかった。name を付けた場合と付けない場合で、40フレーム目の位置が 0.372 / 0.628 / 0.927 / 1.237 …と3桁まで同じ。
- ・ 間隔には狭いほうにも境目がある。高さの 0.25倍 (0.15) では 12枚中3枚で止まった。詰めすぎると、倒れた板が次の板を押し切る前に寝てしまう。
- ・ 広いほうの境目は、高さの 1.03~1.10倍。板は根元を軸に倒れるので、高さぴったりより少し遠くまで届く (厚み 0.08 のぶん)。
- ・ 間隔が広いほど、倒れる列が進む速さは遅い。1秒あたり 10.56枚 (間隔 0.25) → 2.05枚 (0.62)。倒れるまでの回転に時間がかかるため。
- ・ 枚数を増やしても、速さはほぼ同じ。6枚 10.00枚/秒、12枚 7.33枚/秒、24枚 6.57枚/秒。並べる長さを倍にすれば、かかる時間もほぼ倍になる。
- ・ フレーム数が足りないと「途中で止まった」に見える。96フレームで打ち切ったときは、24枚と間隔 0.55 が途中で止まったと出た。160フレームまで伸ばすと最後まで倒れた。止まったのか、まだ続いているのかは、最後のフレームの傾きで見分ける (止まったものは 0~12度、倒れたものは 74度以上)。
- ・ 計算は軽い。12枚・160フレームで 0.5~0.8秒。24枚でも 0.63秒。

次にやること

- ・ 壁を崩す (破片の細かさと時間)
- ・ 曲げて並べたドミノでも、同じ間隔で倒れるか
- ・ 振り返り点検 (061~090)

壁を崩す — 拘束の強さの窓は狭い。弱いと自重で崩れ、強いとぶつけても壊れない

2026-09-24

何を作ったか

3 × 2 × 0.25 の壁を rbdmaterialfracture (Concrete) で割り、半径 0.25 の球を横からぶつけた。破片どうしは、割るときに一緒に作られる拘束でつながっている。

最初は壁がまったく崩れなかった。球は壁に当たって止まり（2.5秒で 1.35 しか進まない）、破片はいちばん動いたものでも 0.006。拘束の強さ (Primary Strength) が既定のままだと、この壁はぶつけても壊れない。

強さを下げていくと崩れ始めた。10000 と 1000 では破片が1つも動かず、100 で16個が動いた。10 / 1 / 0 は結果がまったく同じで、10 より下は「つながっていない」のと変わらない。

ところが 100 では、球を当てなくても崩れた。速さ 0（球は落ちるだけ）でも15個が動く。壁が自分の重さを支えられていない。そこで強さを上げ直すと、500

で「自立はするが、速い球で崩れる」状態になった。速さ 0 で動いた破片は1個（0.155）、速さ 8 で6個、速さ 12 で19個。700 まで上げると、速さ 12 でぶつけても動くのは1個だけだった。

破片の数は、割る時間にはほとんど効かないが、解く時間には効く。10 / 25 / 50 / 100 / 200個で、割るのは 1.10~1.28秒とほぼ一定、解くのは 0.26 / 0.29 / 0.40 / 0.56 / 1.15秒。破片が20倍で、解く時間は4.4倍。

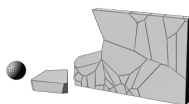
拘束の強さを変えて、同じ球をぶつける

破片 50個、球の速さ 12、58フレーム目。3枚とも同じカメラ。



Houdini

強さ 700。当たったところが1つ外れるだけで、壁は立っている。



Houdini

強さ 500。当たった側の下半分が崩れる。



Houdini

強さ 100。球が届く前から、自分の重さで崩れている。

B. 拘束の強さ（破片 25・球の速さ 8）

「動いた破片」は、60フレームの間に 0.1 以上動いた数。

Primary Strength	動いた破片	いちばん動いた距離	球が進んだ距離
10000	0 / 25	0.006	1.349
1000	0 / 25	0.006	1.349
100	16 / 25	1.157	0.754
10	15 / 25	2.439	0.772
1	15 / 25	2.439	0.772
0	15 / 25	2.439	0.772

自立するか、壊れるか（破片 25）

速さ 0 は「球を当てない」場合。球は落ちるだけで壁には届かない。

強さ	速さ 0（当てない）	速さ 8	速さ 12
100	15個 動く（自重で崩れる）	16個	—
300	15個 動く（自重で崩れる）	20個	—
500	1個（0.155）	6個（0.204）	19個（1.801）
700	0個	1個（0.151）	1個（0.158）
1000	0個	0個	—

A. 破片の数（球の速さ 8・強さ 100）

「割る」は rbdmaterialfracture の計算、「解く」は60フレームのシミュレーション。

指定した数	できた破片	面の数	割る時間	解く時間	動いた破片
10	10	71	1.28秒	0.26秒	3
25	25	216	1.12秒	0.29秒	16
50	50	475	1.10秒	0.40秒	42
100	100	1,015	1.12秒	0.56秒	85
200	200	2,246	1.15秒	1.15秒	179

わかったこと

- ・既定の強さでは、ぶつけても壊れない。Primary Strength 1000 以上では、速さ 8 の球を当てても破片は1つも動かなかった（いちばん動いて 0.006）。
- ・強さを下げすぎると、当てる前に自分の重さで崩れる。100 では、球を当てなくても15個が動いた。「崩れた」を見たら、まず球を当てない場合を試す。
- ・使える窓は狭い。この壁（3 × 2 × 0.25・破片25）では 500 前後。速さ 0 で1個（0.155）、速さ 8 で6個、速さ 12 で19個。700 にすると速さ 12 でも1個しか動かない。
- ・10 より下は、つながっていないのと同じ。強さ 10 / 1 / 0 の結果が、動いた破片15個・いちばん動いた 2.439 まで完全に一致した。
- ・破片の数は、割る時間にはほぼ効かない。10個でも 200個でも 1.10～1.28秒。効くのは解く時間で、0.26秒（10個）→ 1.15秒（200個）。破片が20倍でも4.4倍で済む。
- ・破片を細かくすると、崩れる範囲も広がる。動いた破片の割合は 3/10 → 16/25 → 42/50 → 85/100 → 179/200 と、3割から9割へ増えた。同じ強さなら、細かいほど崩れやすい。
- ・球は破片と同じ形で混ざる。球を packed にして混ぜたときは、球だけが動いて壁は無反応だった。破片は「name で区切られたポリゴン」なので、球にも name を付けて同じ形でつないだら動いた。
- ・計算の時間は、破片25個で割る 1.1秒・解く 0.3秒。破片200個でも解くのは 1.15秒。

次にやること

- ・壁の大きさを変えると、自立できる強さはどう動くか

- ・破片を2段階（Fracture Level 2）にすると、当たったところだけ細かくできるか
- ・振り返り点検（061～090）

voronoifracture は種 N 個で N 個のかげら、体積の合計は元のまま 一切り口を作らないと、中のかげらは消える

2026-09-24

何を作ったか

1×1×1 の箱の中に scatter で N 個の点を撒き (isoffset で中を埋めてから撒いた)、voronoifracture で割った。かけらは name 属性の種類で数え、measure で体積を測った。

**かけらは種の数と同じ N 個、体積を足すとちょうど

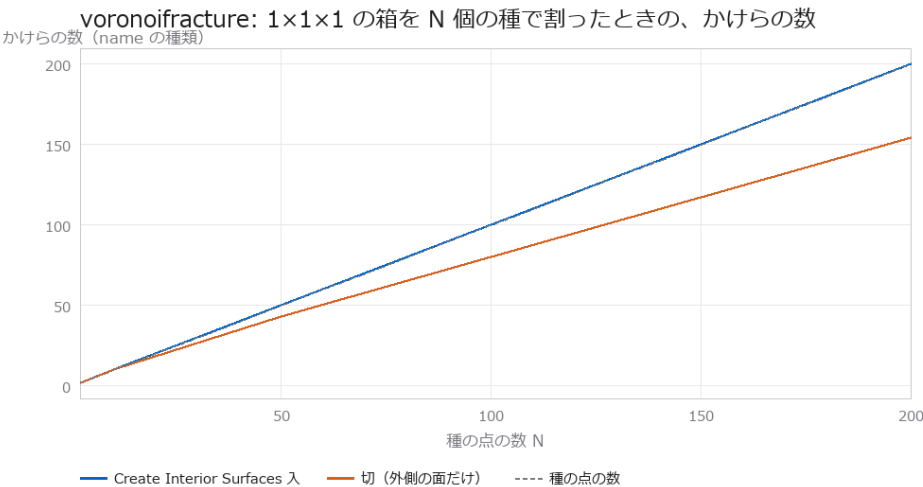
1。 **5通りすべてそうなった (6桁)。すき間も重なりもなく、箱をちょうど分けている。

**かけらの大きさはそろわない。 **N = 50 で、いちばん小さいかけら 0.000098、大きいかけら 0.054061 (552 倍)。平均は $1/N = 0.02$ 。種を撒いた場所しだいが決まる。

**Create Interior Surfaces を切ると、切り口の面が作られない。 **残るのは元の箱の表面を割った面だけで、箱の表面に触れないかけらは消える (N = 50 で 43 個、200 で 154 個)。形が閉じないので体積も測れない。

**速い。 **N = 200 でも 0.047 秒 (1回目だけ 0.151 秒かった)。

種の数・切り口の有無と、かけら



切り口ありは点線 (N) にぴったり乗る。切ると中のかげらの分だけ下がる。

N	切り口	かけら	面	点	体積の合計	最小のかげら	最大のかげら	秒
2	作る	2	12	16	1.000000	0.324096	0.675904	0.1507
2	作らない	2	10	16	—	—	—	0.0120
5	作る	5	36	52	1.000000	0.067480	0.306878	0.0082
5	作らない	5	18	44	—	—	—	0.0072
10	作る	10	86	132	1.000000	0.021517	0.231785	0.0092
10	作らない	10	30	88	—	—	—	0.0077

N	切り口	かけら	面	点	体積の合計	最小のかけら	最大のかけら	秒
50	作る	50	554	908	1.000000	0.000098	0.054061	0.0170
50	作らない	43	84	328	—	—	—	0.0090
200	作る	200	2,378	3,956	1.000000	0.000173	0.014229	0.0470
200	作らない	154	222	1,048	—	—	—	0.0116

わかったこと

- ・かけらの数は種の点の数で決まる。欲しい数だけ点を撒く。
- ・体積は割っても変わらない。密度が同じなら、RBD の質量の合計も元のままのはず（RBD では確かめていない）。
- ・大きさは最大と最小で数百倍ちがう。scatter の relax で種を広げると、かえってばらつく（実験158）。

次にやること

- ・種を relax したときの大きさのばらつき
- ・丸い形（球）を割ったときの体積の合計

種を relax すると、voronoifracture のかけらはそろそろどころかばらつく 一種が箱の壁へ押し出される

2026-09-24

何を作ったか

実験157 と同じ 1×1×1 の箱に scatter で 50 個の種を撒き、Relax Iterations を 0～100 と変えて voronoifracture で割った。Global Seed は 3 通り。かけらの体積の変動係数（標準偏差 ÷ 平均）を比べた。

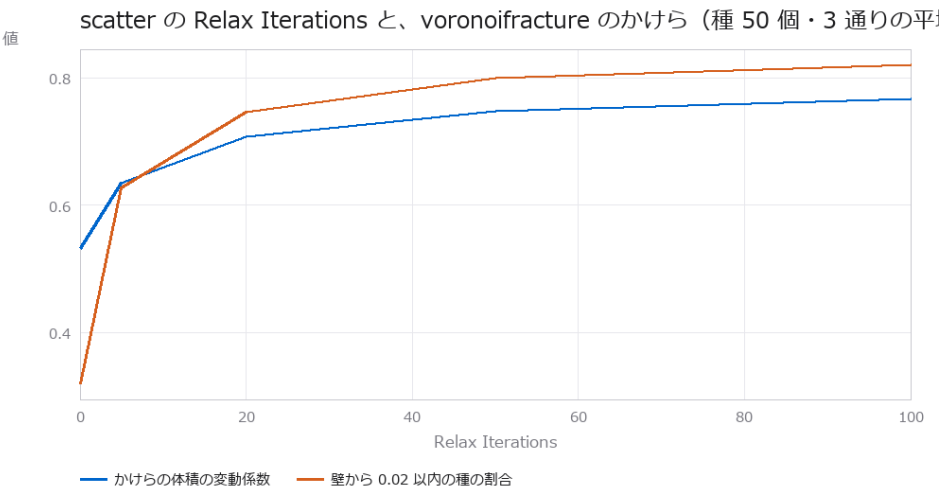
**relax を強くするほど、かけらの大きさはばらついた。 **変動係数は 0 回で平均 0.53、100 回で 0.77。「relax で種を広げればそろそろ」という予想（実験157 の次の課題）は外れた。

**理由: relax は種を箱の壁へ押し出す。 **壁から 0.02 以内にある種は 0 回で平均 32%、100 回で 82%。壁までの距離の平均は、15 通りのうち 6 通りでマイナス（箱の外）になった。壁に張りついた種は、壁ぞいの薄いかけらと、中の大きなかけらを作る。

**種どうしの最短距離は伸びる。 **relax 自体は「点どうしを離す」ことをしている。ただし、壁で止まらない。

**relax なしでも、壁から 0.02 以内の種が約 3 割あった。 **一様に撒けば約 12% のはずで、isoffset で作った中身の範囲が箱より少し広いとみられるが、確かめていない。

Relax Iterations ・ Seed ごとのかけら



変動係数（青）と、壁ぎわの種の割合（橙）が一緒に上がる。

Relax	Seed	かけら	体積の合計	変動係数	最大 ÷ 最小	種の最短距離	壁から 0.02 以内	壁までの平均	秒
0	3	50	1.000000	0.620	23.77	0.0309	34%	+0.0974	0.0911
0	7	50	1.000000	0.513	37.74	0.0423	34%	+0.0859	0.0128
0	11	50	1.000000	0.464	15.41	0.0773	28%	+0.1195	0.0121

Relax	Seed	かけら	体積の合計	変動係数	最大 ÷ 最小	種の最短距離	壁から 0.02 以内	壁までの平均	秒
5	3	50	1.00000 0	0.632	29.39	0.0444	58%	+0.0426	0.0122
5	7	50	1.00000 0	0.696	31.11	0.0940	76%	+0.0004	0.0124
5	11	50	1.00000 0	0.577	14.84	0.1351	54%	+0.0492	0.0125
20	3	50	1.00000 0	0.725	444.58	0.0454	74%	+0.0038	0.0125
20	7	50	1.00000 0	0.729	25.8	0.0940	82%	-0.0124	0.0125
20	11	50	1.00000 0	0.668	118.04	0.1351	68%	+0.0161	0.0119
50	3	50	1.00000 0	0.772	443.72	0.0454	82%	-0.0090	0.0127
50	7	50	1.00000 0	0.761	28.07	0.0940	84%	-0.0175	0.0131
50	11	50	1.00000 0	0.711	103.65	0.1390	74%	+0.0116	0.0122
100	3	50	1.00000 0	0.774	460.26	0.0454	82%	-0.0104	0.0124
100	7	50	1.00000 0	0.758	27.56	0.0940	84%	-0.0175	0.0126
100	11	50	1.00000 0	0.768	125.38	0.1390	80%	-0.0039	0.0140

わかったこと

- ・かけらをそろえる目的で scatter の relax を使うと逆効果。種が壁に寄る。
- ・体積の合計は、relax しても 1 のまま。かけらの数も 50 のまま。
- ・壁の外に出た種もある。割る形の中に種が収まっているか確かめてから使う。

次にやること

- ・箱より小さい範囲に撒いてから relax したとき
- ・pointrelax (relax SOP) で種を広げたとき

rbdbulletsolver の箱は地面ぴったりで止まる — Collision Padding を変えても止まる高さは同じ、当たった瞬間だけ少し沈む

2026-09-24

何を作ったか

1×1×1 の箱（中心 $y = 2$ ）に name を付けて rbdbulletsolver に入れ、Ground Plane ($y = 0$) へ落とした。Collision Padding ($0 \cdot 0.02 \cdot 0.05$) と Bounce ($0.25 \cdot 0.8$ 。地面の Bounce も同じ値) を変え、箱の底（点の y の最小）を毎フレーム読んだ。

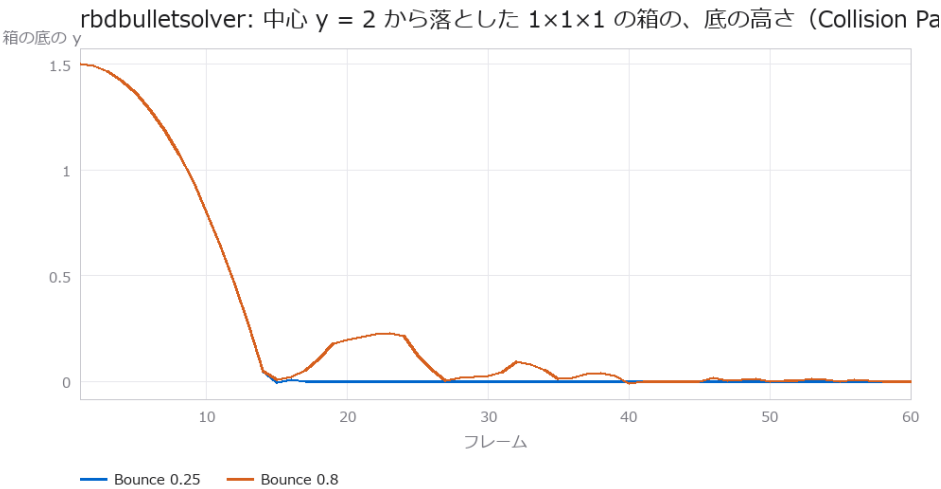
****止まった箱の底は $y = 0$ のすぐ近く。 **4 秒後の底は -0.000092 、中心は 0.499950 。形のまわりに余白を付けるつまみがあっても、見た目の形は地面から浮かない。**

****Collision Padding を $0 \cdot 0.02 \cdot 0.05$ と変えても、結果は6桁まで同じだった****（すべての値が一致）。この形（凸包）では、余白が止まる高さにも跳ね方にも出ない。Bullet が余白の分だけ形を内側へ縮めてから余白を足している可能性があるが、確かめていない。

****当たった瞬間だけ、少し地面にめり込む****（いちばん深くて -0.0082 ）。15 フレーム目に当たる。

****Bounce で跳ね返りの高さが大きく変わる。 **最初の跳ね返りの高さ（底の y ）は Bounce 0.25 で 0.0091 、0.8 で 0.2286 。落とした高さ（底が 1.5 から）に対して、それぞれ $0.6\% \cdot 15\%$ 。**

Collision Padding ・ Bounce と、止まり方



Bounce 0.8（橙）は何度か跳ねてから止まる。0.25（青）はほぼ跳ねない。

Collision Padding	Bounce	止まった底の y	止まった中心の y	いちばん深い所	当たったフレーム	跳ね返りの高さ	97 フレームの秒
0	0.25	-0.000092	0.499950	-0.006908	15	0.0091	0.582
0	0.8	-0.000007	0.499996	-0.008162	15	0.2286	0.294
0.02	0.25	-0.000092	0.499950	-0.006908	15	0.0091	0.291
0.02	0.8	-0.000007	0.499996	-0.008162	15	0.2286	0.288
0.05	0.25	-0.000092	0.499950	-0.006908	15	0.0091	0.291
0.05	0.8	-0.000007	0.499996	-0.008162	15	0.2286	0.295

わかったこと

- ・ 箱は地面ぴったりで止まる。Collision Padding で浮いて見えることはなかった。
- ・ 当たった瞬間は 0.007~0.008 (箱の一边 1 に対して) 沈む。速い物ほど深い可能性がある (確かめていない) 。
- ・ 跳ね返りは Bounce しだい。0.25 ではほとんど跳ねない。

次にやること

- ・ Bullet Substeps を増やしたときの沈み込み
- ・ 球 (Collision Shape = Sphere) での Collision Padding

rbdbulletsolver の沈み込みは Bullet Substeps で決まる — 1 だと 0.38 めり込んで沈んだまま、50 でほぼ 0

2026-09-24

何を作ったか

実験167 の続き。1×1×1 の箱（底は $y = 1.5$ から）を Ground Plane に落とし、Bounce（箱と地面の両方）と Bullet Substeps（既定 10）を変えて、箱の底の y を毎フレーム読んだ。

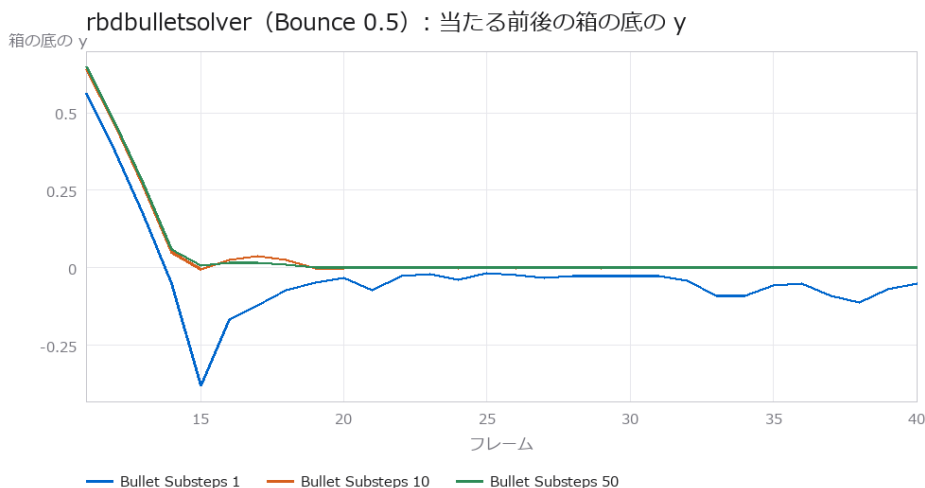
Bullet Substeps 1 では、当たった瞬間に 0.379 めり込み、止まっても 0.064 沈んだままだった。既定の 10 で 0.0050、50 で 0.000008（止まった底は -0）。Substeps を減らして軽くすると、地面の中に沈んだ絵になる。

跳ね返りの高さは、Bounce から想像するよりずっと低い。落とした高さ 1.5 に対して、Bounce $0 \cdot 0.25 \cdot 0.5 \cdot 0.75$ で $0.0011 \cdot 0.0091 \cdot 0.0387 \cdot 0.0860$ （ $1.5 \times \text{Bounce}^2$ なら $0 \cdot 0.094 \cdot 0.375 \cdot 0.84$ ）。ただしフレームごとに読んだ値なので、跳ね返りのいちばん高い所を逃している可能性がある。

Bounce 1 だと、4 秒たっても跳ね続けた（4 秒後の底が 1.87）。エネルギーがほとんど減らない。

時間はどれも 97 フレームで 0.3~0.6 秒。Substeps 50 でも 0.37 秒。

Bounce・Bullet Substeps と、沈み込み・跳ね返り



Substeps 1（青）だけが、当たったあと 0 より下に沈む。

Bounce	Bullet Substeps	いちばん深い所	止まった底の y	跳ね返りの高さ	当たったフレーム	97 フレームの秒
0	10	-0.008692	-0.000104	0.0011	15	0.553
0.25	10	-0.006908	-0.000092	0.0091	15	0.291
0.5	10	-0.005043	-0.000090	0.0387	15	0.288
0.75	10	-0.002914	-0.000010	0.0860	15	0.291
1	10	0.015005	1.868791	0.8294	41	0.287

Bounce	Bullet Substeps	いちばん深い所	止まった底の y	跳ね返りの高さ	当たったフレーム	97 フレームの秒
0.5	1	-0.378997	-0.063566	-0.0170	14	0.280
0.5	50	-0.000008	-0.000000	0.0162	15	0.373

わかったこと

- ・ RBD が地面に沈むなら、まず Bullet Substeps。1 では 0.38 めり込み、そのまま 0.06 沈む。
- ・ 既定の 10 でも、当たった瞬間は 0.005 沈む。アップで見せるなら 50。時間はほぼ変わらない。
- ・ Bounce 1 は止まらない。跳ね続けてほしくないなら 1 未満に。

次にやること

- ・ 跳ね返りの高さを 1 フレームより細かく読む
- ・ 球（回らない形）での Bounce と跳ね返りの関係